

Gemini Flash PDF Chat + Knowledge Base in Next.js

A step-by-step build guide for a first-time RAG project

This guide is written for a student who already has a basic chat app and now needs to add: - PDF upload - PDF text extraction - PDF chat - knowledge-base style retrieval - source citations

It follows a beginner-friendly path first, then shows how to scale it later.

1) What we are building

The app will let a user:

1. Upload one or more PDF files
2. Store those PDFs as knowledge-base documents
3. Extract text from each PDF
4. Split the text into chunks
5. Create embeddings for each chunk
6. Retrieve the most relevant chunks for a user question
7. Send those chunks to Gemini Flash
8. Show the answer with sources

Simple architecture

```
User uploads PDF
↓
Next.js API route saves file
↓
PDF text is extracted
↓
Text is chunked
↓
Embeddings are created
↓
Chunks + embeddings are stored
↓
User asks a question
↓
Question embedding is created
↓
Relevant chunks are retrieved
↓
Gemini Flash answers using only those chunks
```

2) Recommended stack

Use this stack for the first version:

- **Next.js App Router**
- **TypeScript**
- **Tailwind CSS**
- **Prisma**
- **SQLite** for local development, or **Postgres** later
- **pdf-parse** for text extraction
- **Google Gemini API** for:
 - chat answers
 - embeddings

Why this stack?

It is simple enough for a first project and still looks professional in a portfolio.

3) Important Gemini notes

Google's current docs say that:

- Gemini models can process PDF documents
- The Files API is recommended for larger files or when you want to reuse the same file across multiple requests
- Uploaded files are stored for 48 hours
- The PDF limit for the Files API is 50 MB
- File Search is a hosted RAG-style feature that imports, chunks, and indexes data
- Gemini Embedding 2 supports documents, including PDFs, for semantic retrieval workflows

For the first version of this project, we will build the RAG pipeline ourselves so the student understands each step.

4) Project folder structure

```
app/  
  api/  
    chat/route.ts  
    documents/  
      upload/route.ts  
      process/route.ts
```

```
page.tsx
layout.tsx

components/
  PdfUploader.tsx
  DocumentList.tsx
  ChatInterface.tsx
  ChatMessage.tsx
  SourceList.tsx
  EmptyState.tsx

lib/
  gemini.ts
  pdf.ts
  chunk.ts
  embeddings.ts
  retrieve.ts
  storage.ts

prisma/
  schema.prisma

public/
  uploads/
```

5) Install dependencies

```
npm install @google/generative-ai pdf-parse prisma @prisma/client
npm install -D @types/pdf-parse
```

If the project uses the App Router and Tailwind already, that is enough for the UI.

6) Environment variables

Create a `.env` file:

```
GEMINI_API_KEY=your_google_gemini_key
DATABASE_URL="file:./dev.db"
```

If you later move to Postgres, just change `DATABASE_URL`.

7) Database schema

This is the simplest schema for a first version.

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model Document {
  id          String   @id @default(cuid())
  fileName    String
  filePath    String
  status      String   @default("uploaded")
  text        String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  chunks      Chunk[]
}

model Chunk {
  id          String   @id @default(cuid())
  documentId  String
  document    Document @relation(fields: [documentId], references: [id], onDelete: Cascade)
  chunkIndex  Int
  content     String
  embedding   Json
  pageNumber  Int?
  createdAt   DateTime @default(now())
}

model ChatMessage {
  id          String   @id @default(cuid())
  role        String
  content      String
  createdAt    DateTime @default(now())
}
```

What this means

- Document stores each uploaded PDF
- Chunk stores pieces of extracted text plus their embeddings

- ChatMessage is optional, useful if you want chat history
-

8) Create a reusable Prisma client

```
// lib/storage.ts
import { PrismaClient } from "@prisma/client";

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    log: ["error", "warn"],
  });

if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```

9) Gemini helper

```
// lib/gemini.ts
import { GoogleGenerativeAI } from "@google/generative-ai";

const apiKey = process.env.GEMINI_API_KEY;

if (!apiKey) {
  throw new Error("Missing GEMINI_API_KEY");
}

const genAI = new GoogleGenerativeAI(apiKey);

export const chatModel = genAI.getGenerativeModel({
  model: "gemini-1.5-flash",
});

export const embeddingModelName = "text-embedding-004";
```

Hint

Keep Gemini setup in one file. Do not repeat API key code across routes.

10) PDF text extraction helper

```
// lib/pdf.ts
import fs from "fs/promises";
import pdf from "pdf-parse";

export async function extractTextFromPdf(filePath: string) {
  const buffer = await fs.readFile(filePath);
  const data = await pdf(buffer);

  return {
    text: data.text,
    pages: data.numpages,
  };
}
```

Important hint

This works well for text-based PDFs. If the PDF is scanned images, the text may be empty or poor. OCR can be added later.

11) Chunking helper

```
// lib/chunk.ts
export function chunkText(text: string, chunkSize = 1200, overlap = 200) {
  const cleanText = text.replace(/\s+/g, " ").trim();
  const chunks: string[] = [];

  let start = 0;

  while (start < cleanText.length) {
    const end = Math.min(start + chunkSize, cleanText.length);
    const chunk = cleanText.slice(start, end).trim();

    if (chunk) chunks.push(chunk);

    start = end - overlap;
    if (start < 0) start = 0;
    if (end === cleanText.length) break;
  }

  return chunks;
}
```

Why overlap matters

It helps preserve meaning when a sentence is split across chunks.

12) Embedding helper

```
// lib/embeddings.ts
import { GoogleGenerativeAI } from "@google/generative-ai";
import { embeddingModelName } from "../gemini";

const apiKey = process.env.GEMINI_API_KEY;

if (!apiKey) {
  throw new Error("Missing GEMINI_API_KEY");
}

const genAI = new GoogleGenerativeAI(apiKey);
const embeddingModel = genAI.getGenerativeModel({
  model: embeddingModelName,
});

export async function getEmbedding(text: string) {
  const result = await embeddingModel.embedContent(text);
  return result.embedding.values;
}
```

Hint

Generate embeddings once during upload processing. Do not regenerate them every time the user asks a question.

13) Similarity search helper

For a first project, you can store embeddings in the database and compare them in Node.js.

```
// lib/retrieve.ts
export function cosineSimilarity(a: number[], b: number[]) {
  if (a.length !== b.length) return 0;

  let dot = 0;
  let magA = 0;
  let magB = 0;
```

```

    for (let i = 0; i < a.length; i++) {
      dot += a[i] * b[i];
      magA += a[i] * a[i];
      magB += b[i] * b[i];
    }

    const denominator = Math.sqrt(magA) * Math.sqrt(magB);
    if (denominator === 0) return 0;

    return dot / denominator;
  }

export function getTopKMatches(
  queryEmbedding: number[],
  chunks: { id: string; content: string; embedding: any; documentId: string; pageNumber: number },
  k = 5
) {
  return chunks
    .map((chunk) => ({
      ...chunk,
      score: cosineSimilarity(queryEmbedding, chunk.embedding as number[]),
    }))
    .sort((a, b) => b.score - a.score)
    .slice(0, k);
}

```

Hint

This is good for a small student project. If the project grows, move to pgvector, Pinecone, or the Gemini File Search flow.

14) Upload API route

This route saves the file first, then stores a database record.

```

// app/api/documents/upload/route.ts
import { NextRequest, NextResponse } from "next/server";
import fs from "fs/promises";
import path from "path";
import { prisma } from "@/lib/storage";

export const runtime = "nodejs";

export async function POST(req: NextRequest) {
  try {

```



```

const formData = await req.formData();
const file = formData.get("file") as File | null;

if (!file) {
  return NextResponse.json({ error: "No file provided" }, { status: 400 });
}

if (file.type !== "application/pdf") {
  return NextResponse.json({ error: "Only PDF files are allowed" }, { status: 400 });
}

const arrayBuffer = await file.arrayBuffer();
const buffer = Buffer.from(arrayBuffer);

const uploadDir = path.join(process.cwd(), "public", "uploads");
await fs.mkdir(uploadDir, { recursive: true });

const safeName = `${Date.now()}-${file.name}`.replace(/\s+/g, "-");
const filePath = path.join(uploadDir, safeName);

await fs.writeFile(filePath, buffer);

const doc = await prisma.document.create({
  data: {
    fileName: file.name,
    filePath: `/uploads/${safeName}`,
    status: "uploaded",
  },
});

return NextResponse.json({ success: true, document: doc });
} catch (error) {
  console.error(error);
  return NextResponse.json({ error: "Upload failed" }, { status: 500 });
}
}

```

Hint

Keep the upload and processing steps separate. That makes debugging much easier.

15) Process API route

This route extracts text, chunks it, creates embeddings, and stores everything.

```

// app/api/documents/process/route.ts
import { NextRequest, NextResponse } from "next/server";
import path from "path";
import { prisma } from "@/lib/storage";
import { extractTextFromPdf } from "@/lib/pdf";
import { chunkText } from "@/lib/chunk";
import { getEmbedding } from "@/lib/embeddings";

export const runtime = "nodejs";

export async function POST(req: NextRequest) {
  try {
    const { documentId } = await req.json();

    if (!documentId) {
      return NextResponse.json({ error: "documentId is required" }, { status: 400 });
    }

    const doc = await prisma.document.findUnique({ where: { id: documentId } });

    if (!doc) {
      return NextResponse.json({ error: "Document not found" }, { status: 404 });
    }

    const absolutePath = path.join(process.cwd(), "public", doc.filePath.replace(/^\/\//, ""));

    const { text } = await extractTextFromPdf(absolutePath);

    if (!text || !text.trim()) {
      await prisma.document.update({
        where: { id: documentId },
        data: { status: "failed" },
      });

      return NextResponse.json({
        error: "No text could be extracted. This may be a scanned PDF.",
      }, { status: 400 });
    }

    const chunks = chunkText(text);

    await prisma.chunk.deleteMany({
      where: { documentId },
    });

    const chunkRecords = [];

```

```

for (let i = 0; i < chunks.length; i++) {
  const embedding = await getEmbedding(chunks[i]);

  chunkRecords.push({
    documentId,
    chunkIndex: i,
    content: chunks[i],
    embedding,
    pageNumber: null,
  });
}

await prisma.chunk.createMany({
  data: chunkRecords,
});

await prisma.document.update({
  where: { id: documentId },
  data: {
    text,
    status: "processed",
  },
});

return NextResponse.json({
  success: true,
  chunksCreated: chunkRecords.length,
});
} catch (error) {
  console.error(error);
  return NextResponse.json({ error: "Processing failed" }, { status: 500 });
}
}

```

Hint

Later you can add page-number tracking if you extract page by page instead of reading the whole PDF at once.

16) Chat API route

This is the actual RAG endpoint.

```

// app/api/chat/route.ts
import { NextRequest, NextResponse } from "next/server";

```

```

import { prisma } from "@lib/storage";
import { getEmbedding } from "@lib/embeddings";
import { getTopKMatches } from "@lib/retrieve";
import { chatModel } from "@lib/gemini";

export const runtime = "nodejs";

export async function POST(req: NextRequest) {
  try {
    const { question } = await req.json();

    if (!question) {
      return NextResponse.json({ error: "Question is required" }, { status: 400 });
    }

    const queryEmbedding = await getEmbedding(question);

    const chunks = await prisma.chunk.findMany({
      include: { document: true },
    });

    const topMatches = getTopKMatches(queryEmbedding, chunks, 5);

    const context = topMatches
      .map((chunk, index) => {
        return `Source ${index + 1}
Document: ${chunk.document.fileName}
Chunk: ${chunk.content}`;
      })
      .join("\n\n");

    const prompt = `
You are a helpful assistant answering questions from uploaded PDF documents.

Use only the context below.
If the answer is not in the context, say: "I could not find that in the uploaded PDFs."

Context:
${context}

Question:
${question}
`;

    const result = await chatModel.generateContent(prompt);
    const answer = result.response.text();
  }
}

```

```

return NextResponse.json({
  answer,
  sources: topMatches.map((match) => ({
    documentId: match.documentId,
    fileName: match.document.fileName,
    chunkId: match.id,
    score: match.score,
  })),
});
} catch (error) {
  console.error(error);
  return NextResponse.json({ error: "Chat failed" }, { status: 500 });
}
}

```

Hint

The prompt must strongly tell Gemini not to invent answers. This is what keeps the RAG app reliable.

17) Main page layout

Use one page with three areas: - upload section - documents list - chat window

```

// app/page.tsx
import PdfUploader from "@/components/PdfUploader";
import DocumentList from "@/components/DocumentList";
import ChatInterface from "@/components/ChatInterface";

export default function HomePage() {
  return (
    <main className="min-h-screen bg-slate-50 p-6">
      <div className="mx-auto max-w-6xl space-y-6">
        <header className="rounded-2xl bg-white p-6 shadow-sm">
          <h1 className="text-3xl font-bold">PDF Chat Knowledge Base</h1>
          <p className="mt-2 text-slate-600">
            Upload PDFs, process them, and chat with Gemini Flash using retrieved context.
          </p>
        </header>

        <div className="grid gap-6 lg:grid-cols-3">
          <section className="rounded-2xl bg-white p-6 shadow-sm lg:col-span-1">
            <PdfUploader />
            <DocumentList />

```

```

        </section>

        <section className="rounded-2xl bg-white p-6 shadow-sm lg:col-span-2">
          <ChatInterface />
        </section>
      </div>
    </div>
  </main>
);
}

```

18) PDF uploader component

```

// components/PdfUploader.tsx
"use client";

import { useState } from "react";

export default function PdfUploader() {
  const [file, setFile] = useState<File | null>(null);
  const [loading, setLoading] = useState(false);

  async function handleUpload() {
    if (!file) return;

    setLoading(true);
    try {
      const formData = new FormData();
      formData.append("file", file);

      const res = await fetch("/api/documents/upload", {
        method: "POST",
        body: formData,
      });

      const data = await res.json();

      if (!res.ok) throw new Error(data.error || "Upload failed");

      alert("Upload successful. Now process the file.");
      setFile(null);
    } catch (error: any) {
      alert(error.message);
    } finally {

```

```

        setLoading(false);
    }
}

return (
    <div className="space-y-4">
        <h2 className="text-lg font-semibold">Upload PDF</h2>

        <input
            type="file"
            accept="application/pdf"
            onChange={(e) => setFile(e.target.files?.[0] || null)}
            className="block w-full rounded-lg border p-2"
        />

        <button
            onClick={handleUpload}
            disabled={!file || loading}
            className="rounded-lg bg-black px-4 py-2 text-white disabled:opacity-50"
        >
            {loading ? "Uploading..." : "Upload PDF"}
        </button>
    </div>
);
}

```

Hint

After upload, call the process route automatically or show a process button. For a first version, manual process is easier to debug.

19) Document list component

```

// components/DocumentList.tsx
"use client";

import { useEffect, useState } from "react";

type DocumentItem = {
    id: string;
    fileName: string;
    status: string;
};

export default function DocumentList() {

```

```

const [documents, setDocuments] = useState<DocumentItem[]>([]);

useEffect(() => {
  loadDocuments();
}, []);

async function loadDocuments() {
  const res = await fetch("/api/documents");
  const data = await res.json();
  setDocuments(data.documents || []);
}

async function processDoc(documentId: string) {
  await fetch("/api/documents/process", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ documentId }),
  });

  loadDocuments();
}

return (
  <div className="mt-8 space-y-3">
    <h2 className="text-lg font-semibold">Documents</h2>

    {documents.length === 0 ? (
      <p className="text-sm text-slate-500">No PDFs uploaded yet.</p>
    ) : (
      documents.map((doc) => (
        <div key={doc.id} className="rounded-lg border p-3">
          <p className="font-medium">{doc.fileName}</p>
          <p className="text-sm text-slate-500">Status: {doc.status}</p>

          {doc.status === "uploaded" && (
            <button
              onClick={() => processDoc(doc.id)}
              className="mt-2 rounded-md bg-slate-900 px-3 py-1 text-sm text-white"
            >
              Process
            </button>
          )}
        </div>
      ))
    )}
  </div>
)

```



```
    );  
  }  
}
```

20) Chat interface component

```
// components/ChatInterface.tsx  
"use client";  
  
import { useState } from "react";  
  
type Message = {  
  role: "user" | "assistant";  
  content: string;  
};  
  
export default function ChatInterface() {  
  const [messages, setMessages] = useState<Message[]>([]);  
  const [question, setQuestion] = useState("");  
  const [loading, setLoading] = useState(false);  
  
  async function sendQuestion() {  
    if (!question.trim()) return;  
  
    const userMessage: Message = { role: "user", content: question };  
    setMessages((prev) => [...prev, userMessage]);  
    setLoading(true);  
  
    try {  
      const res = await fetch("/api/chat", {  
        method: "POST",  
        headers: { "Content-Type": "application/json" },  
        body: JSON.stringify({ question }),  
      });  
  
      const data = await res.json();  
  
      if (!res.ok) throw new Error(data.error || "Chat failed");  
  
      setMessages((prev) => [  
        ...prev,  
        { role: "assistant", content: data.answer },  
      ]);  
  
      setQuestion("");  
    }  
  }  
}
```

```

    } catch (error: any) {
      setMessages((prev) => [
        ...prev,
        { role: "assistant", content: error.message },
      ]);
    } finally {
      setLoading(false);
    }
  }
}

return (
  <div className="flex h-full flex-col gap-4">
    <h2 className="text-lg font-semibold">Chat with your PDFs</h2>

    <div className="min-h-[400px] space-y-3 rounded-xl border p-4">
      {messages.map((msg, index) => (
        <div
          key={index}
          className={`rounded-lg p-3 ${
            msg.role === "user" ? "bg-slate-100" : "bg-blue-50"
          }`}
        >
          <p className="text-sm font-semibold capitalize">{msg.role}</p>
          <p className="mt-1 whitespace-pre-wrap text-sm">{msg.content}</p>
        </div>
      ))}
    </div>

    <div className="flex gap-2">
      <input
        value={question}
        onChange={(e) => setQuestion(e.target.value)}
        placeholder="Ask something from the uploaded PDFs..."
        className="flex-1 rounded-lg border p-3"
      />
      <button
        onClick={sendQuestion}
        disabled={loading}
        className="rounded-lg bg-black px-4 py-2 text-white disabled:opacity-50"
      >
        {loading ? "Sending..." : "Send"}
      </button>
    </div>
  </div>
);
}

```

21) Documents list API

```
// app/api/documents/route.ts
import { NextResponse } from "next/server";
import { prisma } from "@/lib/storage";

export async function GET() {
  const documents = await prisma.document.findMany({
    orderBy: { createdAt: "desc" },
  });

  return NextResponse.json({ documents });
}
```

22) Add source display

A good RAG app should show where the answer came from.

```
// components/SourceList.tsx
type Source = {
  fileName: string;
  score: number;
};

export default function SourceList({ sources }: { sources: Source[] }) {
  if (!sources.length) return null;

  return (
    <div className="mt-4 rounded-xl border p-4">
      <h3 className="font-semibold">Sources</h3>
      <ul className="mt-2 space-y-1 text-sm">
        {sources.map((source, index) => (
          <li key={index}>
            {source.fileName} - relevance {source.score.toFixed(2)}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Hint

This makes the app feel real and helps during debugging.

23) Optional better version: process immediately after upload

If the student wants a smoother UX:

1. Upload file
2. Save document record
3. Immediately call process route
4. Show a spinner until processing is done

This feels more polished, but for a first project, a separate process button is easier.

24) What to do when the PDF is scanned

If text extraction returns nothing or too little:

- the PDF may be image-based
- add OCR later
- show a message: “This file seems scanned; OCR support is needed”

Do not block the whole project because of OCR. Ship the text-PDF version first.

25) Good prompts for Gemini Flash

Use prompts like this:

`Answer only from the provided context.`

`If the answer is not present, say you cannot find it in the uploaded PDFs.`

`Do not guess.`

`Be concise and clear.`

For better results, include: - document names - chunk text - source numbers

26) Testing checklist

After every major step, test this:

- Upload PDF works

- File is stored
 - Document appears in the list
 - Process button works
 - Chunks are saved
 - Embeddings are saved
 - Chat question returns answer
 - Answer uses PDF context
 - Sources are displayed
 - App does not crash on bad files
-

27) Common problems and fixes

Problem: “No text extracted”

Reason: - scanned PDF - empty PDF - bad extraction library result

Fix: - test with a normal text-based PDF first

Problem: “Model answers incorrectly”

Reason: - wrong chunks retrieved - prompt too loose

Fix: - inspect retrieved chunks before blaming the model - make the prompt stricter

Problem: “App is slow”

Reason: - embeddings are generated too often - everything is processed on every request

Fix: - generate embeddings only during upload processing - reuse stored chunks

Problem: “Too many files get messy”

Fix: - add document categories later - add delete and reprocess buttons

28) Best final project flow for the student

Minimum lovable version

- upload PDF
- process PDF
- chat with PDF
- show sources

Good portfolio version

- multiple PDFs
- document list
- status tracking
- source citations
- clean UI
- loading states
- error states

Advanced version

- OCR
 - multiple users
 - folder/category filters
 - re-ranking
 - pgvector or hosted vector DB
 - chat history persistence
-

29) Suggested implementation order

Do it in this order:

1. Create the schema
 2. Add upload route
 3. Add PDF extraction helper
 4. Add chunking
 5. Add embeddings helper
 6. Add processing route
 7. Add retrieval helper
 8. Add chat route
 9. Build UI components
 10. Add sources and polish
-

30) Final note for the student

Do not try to build the perfect RAG app first.

First goal: - one PDF - one question - one answer

Second goal: - many PDFs

Third goal: - cleaner retrieval and better UX

That sequence makes the project finishable.

Reference notes

Current Gemini docs: - Document processing / PDF support - Files API - File Search - Embeddings

These docs are useful when you later upgrade this project from a custom RAG pipeline to a hosted retrieval solution.